Unsupervised Entity Resolution With Blocking and Graph Algorithms

Dongxiang Zhang, Dongsheng Li, Long Guo, and Kian-Lee Tan, *Senior Member, IEEE*

Abstract—Entity resolution identifies all records in a database that refer to the same entity. In this paper, we propose an unsupervised framework for entity resolution using blocking and graph algorithms. The records are partitioned into blocks with no redundancy for efficiency improvement. For intra-block data processing, we propose a graph-theoretic fusion framework with two components, namely ITER and CliqueRank. Specifically, ITER constructs a weighted bipartite graph between terms and record-record pairs and iteratively propagates the node salience until convergence. Subsequently, CliqueRank constructs a record graph to estimate the likelihood of two records resident in the same clique. The derived likelihood from CliqueRank is fed back to ITER to rectify the edge weight until a joint optimum can be reached. Experimental evaluation was conducted with 4 real datasets. Results show that our unsupervised framework is comparable or even superior to state-of-the-art deep learning approaches.

Index Terms—Unsupervised entity resolution, blocking, graph-based algorithm

1 Introduction

Entity resolution, also known as duplicate record detection, is a fundamental problem in data integration and data cleaning. Given a database with ambiguous and dirty records, it identifies clusters of records referring to the same real-world entity. The problem has a very long research history and various types of methods have been proposed.

Early research works [1], [2], [3] have been dedicated to devise various string-based distance functions, either character-based [1] or token-based [2], to measure pair-wise record similarity. Two items refer to the same entity if their textual similarity exceeds a predefined threshold. However, the effectiveness of these unsupervised approaches is limited. In addition, as emphasized by Bilenko *et al.* [3], there does not exist a unified metric for all data sets.

To overcome the limitations of unsupervised approaches, machine learning based techniques [4], [5], [6], [7] became popular and improved the accuracy to certain extent. The entity resolution problem is viewed as a binary classification task. With hand-crafted features, standard classifiers such as SVM or naive Bayesian can be directly applied. More recently, researchers have also utilized the power of deep learning (DL) models. DeepER [8] adopts Glove [9], a pre-trained language model, for word embedding. These generated word vectors are used as the input to a vanilla LSTM model whose output is considered as the representation of the text record. DeepMatcher [10] is another recent work using recurrent neural network and attention mechanism for entity matching. Its findings indicate that DL-based models significantly outperform current solutions.

With the availability of affordable crowd workers, an altertive research branch focuses on utlizing human intelligence to help verify uncertain pairs [11], [12], [13], [14], [15], [16]. These methods use machines to conduct an initial and coarse filtering based on a simple distance measure to remove pairs unlikely to match. For instance, Jaccard similarity was used in [13], [15] and a threshold of 0.3 was set to filter unpromising pairs. Then, crowd workers were used to verify the most promising pairs. To avoid enumerating all record-pairs to the crowd, which can be prohibitively expensive and time-consuming, these methods focused on devising optimal strategies to select the record-pairs that are most worthy of verification. The objective is to minimize the number of questions asked while achieving high accuracy. In this paper, we consider DL-based models published in recent two years as state-of-the-art approaches and compare with them in the experimental evaluation. This is because the crowdsourcing methods have only been evaluated in small-scale datasets and their effectiveness on larger-scale dataset remains unknown.

Table 1 summarizes the comparison of existing approaches. In this paper, we focus on devising an accurate and unsupervised entity resolution approach that does not require preparing a labelled training dataset for supervised learning or paying crowd workers to verify the enumerated candidate pairs. We propose a non-redundant blocking strategy to partition the whole dataset into blocks with similar size. The records located in different blocks are considered as non-matching. Hence, the blocking mechanism can significantly

- *D. Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou, Zhejiang 310027, China. E-mail: zhangdongxiang37@gmail.com.*
- *D. Li is with the School of Computer Science, National University of Defense Technology, Changsha, Hunan 410073, China. E-mail: dsli@nudt.edu.cn.*
- *L. Guo is with the Alibaba Group, Hangzhou 311121, China. E-mail: leo.gl@alibaba-inc.com.*
- *K.-L. Tan is with the School of Computing, National University of Singapore, Singapore119077. E-mail: tankl@comp.nus.edu.sg.*

Manuscript received 5 Apr. 2019; revised 9 Apr. 2020; accepted 17 Apr. 2020. Date of publication 28 Apr. 2020; date of current version 3 Feb. 2022. (Corresponding author: Dongsheng Li.) Recommended for acceptance by S. Sadiq. Digital Object Identifier no. 10.1109/TKDE.2020.2991063

1041-4347 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Summary of Existing Entity Resolution Approaches

	Labeling	Human Intervention	Accuracy
Previous unsupervised	no	no	not high
Traditional ML	yes	no	not high
Deep learning	yes	no	high
Crowdsourcing	no	yes	high
Proposed	no	no	high

reduce running time as we only need to examine the pairs within the same block.

For intra-block record matching, we propose a new graph-theoretic fusion framework that provides an accurate estimation of the probability of two records referring to the same entity. We observed that, in a specific domain, terms usually exhibit different discrimination power, which should be taken into account in the textual similarity measure. For instance, terms about brand and model are more important than others in distinguishing E-commerce products, while telephone number and address are more discriminative for restaurants. An "intelligent" algorithm should be able to automatically differentiate the importance of terms from data, and consequently, provide high-quality entity resolution. It is worth noting that even though IDF (inverse document frequency) can also set higher weight to rare terms such as product models, the measure is not able to differentiate the truly discriminative terms and noisy terms with low frequency in the dataset. This observation motivated us to propose an enhanced content-based similarity measure that can accurately identify entities by discriminating the importance of terms from data; and most importantly, our method works in a fully unsupervised way. We first construct a new type of bipartite graph to model the relationship between terms and record–record pairs, developing an *Iterative Term-Entity Ranking* (ITER) algorithm to jointly estimate the importance of terms and similarity of record-pairs.

Our next contribution is to derive the confidence of two records belonging to the same entity based on the topological property. In case of an ideal textual similarity measure, two records r_i and r_j referring to different entities should be located at different cliques, which are unreachable from each other. On the contrary, if r_i and r_j refer to the same entity, they should be in the same clique and reachable from each other. From the random-walk perspective, if we start a random walk from one record r_i, it will be very likely to reach the other record r_j within a fixed number of steps. We leverage this key insight to develop CliqueRank to identify the matching probability. This procedure can be viewed as a normalization step that transforms similarity scores to matching property with the range [0,1], from which it is easier to determine a universal threshold for different domains to judge whether two records are matching.

Finally, the enhanced textual similarity and pair matching probability can reinforce each other in our fusion framework. Considering that the matching probability of record-pairs is helpful to determine the discrimination power of terms — for example, the shared terms of non-matching pairs should be penalized with a lower importance and the discriminative terms in matching pairs should be promoted — we further feed the output of CliqueRank into the ITER algorithm. As such, our final solution iterates ITER and CliqueRank algorithms until a stable state is reached. We conduct extensive experiments on four real datasets that are publicly accessible and used in recent DL-based approaches. Without any labelled data or crowd-assistance, our unsupervised framework can achieve comparable or superior performance over the state-of-the-art DL-based models. All the datasets and implementations of our proposed algorithms are publicly accessible at Github.[1]

This manuscript is an extension of our previous conference work published in ICDE 2018 [17], which only evaluates the performance on three small-scale datasets. In this journal, we propose new techniques to extend the work to support datasets in much larger scale. The remaining of the paper is organized as follows. Section 2 discusses related work. We propose our unsupervised fusion framework in Section 3. The bipartite graph model and ITER algorithm are presented in Section 4. In Section 5, we present the CliqueRank algorithm. The reinforcement between ITER and CliqueRank is presented in Section 6. We propose a new blocking strategy in Section 7. Extensions to support structured datasets are presented in Section 8. Comprehensive evaluation and experimental result analysis are conducted in Section 9. We conclude the paper in Section 10.

2 Related Work

Due to space limit, we only review resolution methods that are distance-based, learning-based and crowd-based. A comprehensive survey on this topic is available in [18].

2.1 Distance-Based Methods

The early efforts emphasized on finding or developing a string-based distance (or similarity) metric to detect duplicate records. For instance, edit distance was used in [1] and TF-IDF distance metric was proposed in [2]. Cohen *et al.* conducted an experimental study to compare several types of distance metrics for name-matching tasks [19]. Two items are considered to be identical entity if their textual similarity exceeds a predefined threshold. Even though these approaches are simple and scalable, they are not effective in practice. In addition, they are not adaptive: no single metric is suitable for all data sets [3] and it is non-trivial for the distance-based techniques to define an appropriate matching threshold.

2.2 Learning-Based Methods

To overcome the limitations of unsupervied distance-based methods, supervised learning was used to learn the parameters of string-edit distance metrics [20], [21] or combine the results from different distance metrics [22], [23]. In [5], the entity resolution problem was modeled as a classification task and solved with SVM. One severe problem with the supervised learning techniques is the requirement for a large number of training instances. In addition, the number of non-matches far exceeds the number of matches, which also brings challenges in preparing the training dataset. In subsequent efforts, active learning was adopted to reduce the number of instances required for the classifiers without

1. https://github.com/uestc-db/Unsupervised-Entity-Resolution

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

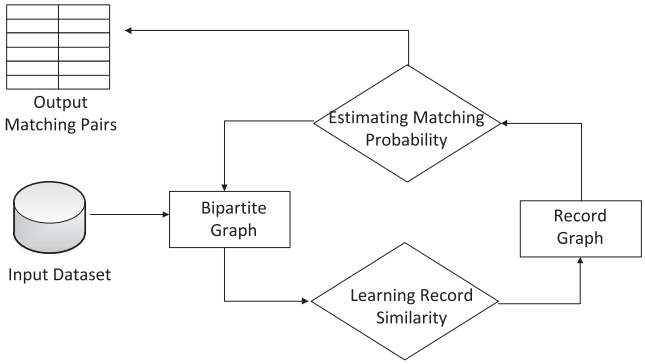


Fig. 1. An overview of our framework for unsupervised entity resolution. We will propose ITER to learn record similarity and CliqueRank to estimate matching probability.

remarkable accuracy degradation [4]. In [24], Bellare *et al.* further improved the active learning framework with provably sub-linear label complexity. Magellen [25] is an end-to-end entity resolution system that covers the entire pipeline. Recent breakthrough in this research area relies on the power of deep learning models, which can automatically learn discriminative features. DeepER [8] adopts Glove [9] for word embeddings, which are used as the input to a vanilla LSTM model. DeepMatcher [10] also uses recurrent neural network, but with additional attention mechanism, for feature generation. The output vector of the DL models are sent to a binary classifier for entity matching.

2.3 Crowd-Based Methods

Recent research interests on this topic have shifted to crowd-assisted solutions in the database community. CrowdER [11] was the first work to systematically adopt a hybrid human-machine approach in which machines were used for a coarse filtering based on simple string-based distance metrics, and then human intelligence was leveraged for verification of the promising but uncertain record-pairs. ZenCrowd [26] is an alternative effort to combine both probabilistic reasoning and crowdsourcing techniques. Although CrowdER and ZenCrowd can achieve a relatively promising precision, they incur considerable amount of crowdsourcing overhead. Several other works [12], [13], [14] have attempted to reduce the number of verification tasks sent to the crowd but this is done at the expense of accuracy. The state-of-the-art work [15] that achieves the highest accuracy is an adaptive approach which uses correlation clustering to reduce the financial costs and improve the accuracy. Following this work, Chai *et al.* proposed a partial-order approach [16] to significantly reduce the candidate pairs delivered for crowd-task while not sacrificing the quality. Fan *et al.* studied the problem of entity collection with the help of crowd workers [27].

3 Unsupervised Fusion Framework

In this section, we propose a novel fusion framework for intra-block pair matching, which takes into account content-based similarity and topological structure simultaneously. Our fusion framework exhibits three distinctive features:

1) We are interested in incorporating term discrimination power when measuring textual similarity. Note that these two ranking schemes can be conflictive in certain cases and details will be explained in Section 4.
2) We leverage the topological structure from a novel perspective. In particular, we construct a weighted record graph with the objective of estimating the *probability* of two records referring to the same entity.
3) Our proposed fusion framework allows the content-based similarity and the pair matching probability to reinforce each other and improve the accuracy of entity resolution.

Fig. 1 summarizes the workflow of our proposed method. We first construct a new bipartite graph to model the relationship between terms and *record–record pairs* and develop an Iterative Term-Entity Ranking (ITER) algorithm to jointly estimate the discrimination power of terms and textual similarity of record-pairs. With the pair similarity scores derived by the ITER algorithm, we construct another weighted record graph with the objective of leveraging its topological structure to estimate the probability of two records belonging to the same entity. Intuitively, a group of matching records should share high similarity scores while non-matching records have low similarity. In other words, if we start a random walk process from one record r_i and choose the next node according to the similarity score, the matching probability can be explained as how likely we can reach a matching record r_j within a fixed number of steps. This observation renders us to propose the CliqueRank algorithm to estimate the matching probability via graph-theoretic approaches.

Finally, the enhanced content-based similarity and pair matching probability can reinforce each other in our fusion framework. The shared terms of non-matching pairs should be penalized with a lower importance as they appear in many pairs with low matching probability. The weight of discriminative terms that only occur in matching pairs would be promoted because these pairs are likely to have high similarity scores and their estimated matching probability would be close to 1. Our fusion framework iterates ITER and CliqueRank algorithms until stability is reached for the update of term weight and record similarity.

It is also worth noting that we can directly use the matching probability, naturally ranged in [0,1], to determine whether two records belong to the same entity. Such a new criterion is more universal and straightforward to explain.

4 Learning Record Similarity

Instead of manually crafting a string-based distance measure, we propose an unsupervised computational method that automatically learns the similarity of records from data. We begin by defining an enhanced content-based similarity measure based on discrimination power, followed by constructing a bipartite graph that models the relationship between terms and record-pairs. After that, we elaborate the ITER algorithm that simultaneously estimates the importance of terms for entity resolution and the similarity of record-pairs. Finally, we prove the convergence of our ITER algorithm. The notations frequently used in this paper are summarized in Table 2.

4.1 Enhanced Content-Based Similarity

The conventional TF-IDF measure assigns higher weight to frequent terms in a document (record) and penalizes

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Notation Table for Common Symbols Used in This Paper

r_i	text record containing a set of terms
n	number of records in the dataset
m	number of unique terms in the dataset
$s(r_i, r_j)$	similarity score between r_i and r_j
$p(r_i, r_j)$	probability of r_i and r_j referring to the same entity
$p(r_i \rightarrow r_j)$	the probability of stepping to r_j in random walk
x_t	term weight or discrimination power of t
N_t	number of records containing term t
P_t	pairs of records sharing term t, i.e., $P_t = N_t \cdot (N_t - 1)/2$
η	matching probability threshold
α	exponent in the non-linear normalization of random walk
M	number of random walk based sampling in RSS
S	maximum number of steps in a random walk
B	number of blocks

common or stop words using the factor of inverse document frequency [28]. Similarly, the TW-IDF measure [29] also prefers frequent terms in the record corpus as they would co-occur with many other terms in the sliding window and serve as hubs in the crafted term graph. However, we observed that discriminative terms, that are really important when determining matching pairs, may not have high TF scores. For example, the term frequency of product models in the dataset such as "pslx350h" and "tu1500rd" may occur only once in a record, but they are highly discriminative in judging the identicalness of two products. Careful readers may argue that the factor of IDF in TF-IDF or TW-IDF can also help promote the weight of these product model terms. However, IDF is a general measure and cannot differentiate the truly discriminative terms and noisy terms with low frequency in the dataset.

In this paper, we propose an enhanced content-based similarity measure based on the discrimination power of terms. The intuition is that if a term only occurs in a group of matching records, then we consider the term as highly discriminative. Examples include product models for electronic devices and telephone numbers for restaurants. On the contrary, a common term without discrimination power is likely to be shared by many non-matching record pairs with low similarity score and matching probability. These two properties essentially differentiate our similarity measure from TF-IDF or TW-IDF. Let n be the number of records in a dataset and suppose term t is contained in a set of N_t records $\{\mathrm{r}_1^{\mathrm{t}}, \mathrm{r}_2^{\mathrm{t}}, \ldots, \mathrm{r}_{\mathrm{N_t}}^{\mathrm{t}}\}$. We formally define our novel definition of the term weight x_t and record similarity score $s(r_i, r_j)$ in a recursive manner

$$\mathrm{x_t} = \sum_{\mathrm{i} \neq \mathrm{j}} \frac{\mathrm{p(r_i, r_j) s(r_i, r_j)}}{\mathrm{P_t}} \tag{1}$$

$$\mathrm{s(r_i, r_j)} = \sum_{t \in r_i \wedge t \in r_j} \mathrm{norm(x_t)}, \tag{2}$$

where there involve $P_t = \frac{N_t \cdot (N_t - 1)}{2}$ pairs with different records and $p(r_i, r_j)$ indicates the probability of r_i and r_j $(i \neq j)$ referring to the same entity. This probability, which will be introduced in Section 5, is initialized to 1 before it can be derived from CliqueRank. In Equation (1), a high x_t is assigned to term t (e.g., "pslx350h" and "tu1500rd") if all the record pairs sharing the term refer to the same entity. In other words, these pairs have high similarity score $s(r_i, r_j)$ and high matching confidence $p(r_i, r_j)$ at the same time. The denominator P_t is a normalization factor to penalize frequently shared terms that are not discriminative (e.g., "dollar" and "device"). Recursively, the record similarity measure $s(r_i, r_j)$ in Equation (2) is defined in a straightforward fashion as the sum of normalized term weight. Based on the definition, $s(r_i, r_j)$ is set to 0 if r_i and r_j do not share any term. This is reasonable because we resolve entities from text-informative records. Each matching pair is likely to share a considerable number of discriminative terms.

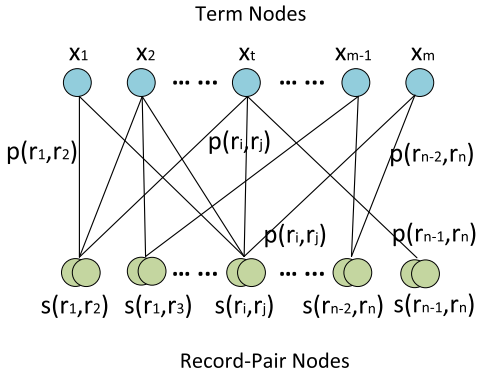


Fig. 2. An example of bipartite graph for terms and record-pairs.

4.2 Bipartite Graph Model

Intuitively, if two records share more discriminative terms, they are more likely to refer to the same entity; and vice versa. To capture this, we use a bipartite graph to capture the relationship between terms and record pairs. Fig. 2 shows an example, where there are two types of nodes — one is *term node* and the other is *pair node*, which denotes a pair of records.

Let t be a term node and (r_i, r_j) be a pair node. Nodes t and (r_i, r_j) are connected if and only if term t appears in both records of r_i and r_j. If a pair (r_i, r_j) does not share any term, we consider the pair of records not matching and exclude it from the bipartite graph. We associate a weight parameter for each node. For a term node t, its weight x_t indicates its power in discriminating entities; for a pair node (r_i, r_j), its weight $s(r_i, r_j)$ denotes the similarity score for the corresponding pair — the higher $s(r_i, r_j)$ is, the more likely pair (r_i, r_j) represents the same entity.

The two types of nodes are connected via weighted edges. The weight $p(r_i, r_j)$ is the estimated matching probability between r_i and r_j. It controls the amount of weight transferred from pair nodes to term nodes. If two records r_i and r_j are not matching, then we consider their shared terms are not contributing for this pair and their $s(r_i, r_j)$ will not be propagated back to adjust the term weight.

4.3 Iterative Term-Entity Ranking Algorithm

The objective of ITER algorithm is to estimate the weights of nodes, i.e., x_t and $s(r_i, r_j)$ from the bipartite graph model tailored for entity resolution. As the sharing of discriminative terms is a strong indicator of the same entity, it is reasonable to infer the weight propagation from term nodes to

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

pair nodes by summing up the node weights of term nodes. This is equivalent to our recursive definition in Equation (1). This equation captures our intuition that the more discriminative terms two records share, the more likely they refer to the same entity.

Likewise, the discrimination power of a term is reflected in its connected pair nodes — if all connected pairs refer to the same entity, then the term is a discriminative feature (an example is product model); on the contrary, if the connected pairs represent a diverse set of entities, then the term is less discriminative (an example is stop words). Mathematically, the propagation from pair nodes to term nodes is equivalent to our definition in Equation (1).

Without any external knowledge, $p(r_i, r_j)$ can be set uniformly for all pair nodes. In our solution, $p(r_i, r_j)$ has another functionality — serving as a bridge to connect our ITER and CliqueRank algorithm, which is detailed later in Section 5. Initially, ITER runs with a uniform $p(r_i, r_j)$ until convergence; then CliqueRank refines $p(r_i, r_j)$ and ITER runs with the new $p(r_i, r_j)$ as the edge weight in the bipartite graph. Equations (1) and (2) naturally form an iterative process for estimating the weights of nodes. Since Equation (2) is an additive rule that increases $s(r_i, r_j)$ without bounds, it is instructive to normalize the term weights x_t in each iteration for the sake of convergence. In our implementation, we set $x_t = x_t/(x_t + 1)$ to normalize the weight between 0 and 1. Other normalization methods such as L_2 norm: $\sum_t x_t^2 = 1$ can also be applied. We summarize the iterative method in Algorithm 1.

Algorithm 1. ITER Algorithm

Input: Bipartite graph structure with edge weight $p(r_i, r_j)$;
Output: Node salience x_t and record pair similarity score $s(r_i, r_j)$;

1: Randomly initialize x_t in (0,1);
2: **while** x_t does not converge **do**
3: **for** each record pair (r_i, r_j) **do**
4: Set its weight $s(r_i, r_j) \leftarrow \sum_{t \in r_i \wedge t \in r_j} x_t$;
5: **for** each term t **do**
6: Set its weight $x_t \leftarrow \sum_{i \neq j} \frac{p(r_i, r_j) s(r_i, r_j)}{P_t}$;
7: Set $x_t = 1/(1 + \frac{1}{x_t})$
8: **return** x_t and $s(r_i, r_j)$

The key difference of our algorithm with PageRank based approaches is in Equation (1), where the denominator penalizes the score of x_t by its number of connected nodes. We point out that this key difference is crucial to estimate the discriminative power of a term. More specifically, if t is a common term and occurs in many entries (such as some domain-specific stop words), its weight will be diluted by the non-matching pairs. Since PageRank lacks the denominator for penalizing frequent terms, it will assign a high score to common terms, thus failing to estimate the salience of a term for entity resolution. The other difference is that we introduce $p(r_i, r_j)$ as the edge weight to promote terms only occurring in matching pairs.

Complexity and Space Analysis. Let l_m be the maximum number of terms in a record. The memory cost of ITER is determined by the bipartite graph. We need to store two types of nodes whose cost is $O(m + n(n-1)/2)$, where m is the number of unique terms and $n(n-1)/2$ is the number of record pairs. Since each pair of record shares at most l_m terms, the number of edges is $O(l_m n(n-1)/2)$. The running time of ITER algorithm grows quadratically with the number of input records. In each iteration, all the edges are traversed twice to update x_t and $s(r_i, r_j)$ respectively. From our experimental results, we observed that the algorithm could quickly reach convergence with only a few iterations.

4.4 Proof of Convergence

Let matrix $\mathbf{S}$ denote the adjacency matrix of the bipartite graph, where each entry $S_{ab} = 1$ if and only if there is an edge between term t_a and pair p_b; otherwise, S_{ab} is 0. Here, we represent b as a unique index for each pair (r_i, r_j) and set $b = i * n + j$. Let vector $\mathbf{x}$ and $\mathbf{y}$ denote the weight vector for term nodes and pair nodes, respectively. Then, we can rewrite the iterative update rules in the matrix form

$$\mathbf{y} = \mathbf{S}^T\mathbf{x}; \qquad \mathbf{x} = \mathbf{D}^{-1}\mathbf{S}\mathbf{C}\mathbf{y}, \tag{3}$$

where $\mathbf{D}$ is a diagonal matrix with each entry $D_{aa} = P_{t_a}$ $(1 \leq a \leq m)$, and $\mathbf{C}$ is also a diagonal matrix with each entry $C_{bb} = p(r_i, r_j)$ for $b = i * N + j$. In the next, we show the convergence of the $\mathbf{y}$ vector, as $\mathbf{x}$ can be directly derived from $\mathbf{y}$.

Theorem 1. *The* $\mathbf{y}$ *vector of iterations (defined in Equation (3)) converges to stationary solution* $\mathbf{y}^*$.

Proof. First, we eliminate $\mathbf{x}$ in $\mathbf{y}$'s update rule

$$\mathbf{y}^{(k)} = (\mathbf{S}^T\mathbf{D}^{-1}\mathbf{S}\mathbf{C})\mathbf{y}^{(k-1)} = \cdots = (\mathbf{S}^T\mathbf{D}^{-1}\mathbf{S}\mathbf{C})^k\mathbf{y}^{(0)}, \tag{4}$$

where k denotes the number of iterations, and $\mathbf{y}^{(0)}$ denotes the initialization of $\mathbf{y}$. By standard linear algebra [30], if $\mathbf{M}$ is a diagonalizable matrix, and $\mathbf{v}$ is a vector not orthogonal to the principle eigenvector of $\mathbf{M}$, then the direction of $\mathbf{M}^k v$ converges to the principle eigenvector as k increases without bound. Since $\mathbf{S}^T\mathbf{D}^{-1}\mathbf{S}$ is a symmetric matrix and $\mathbf{C}$ is diagonal, $\mathbf{S}^T\mathbf{D}^{-1}\mathbf{S}\mathbf{C}$ is guaranteed to be a diagonalizable matrix. Thus, the iterative algorithm finally converges to the stationary solution:

$$\mathbf{y}^* = \mathbf{w}_1; \qquad \mathbf{x}^* = \mathbf{D}^{-1}\mathbf{S}\mathbf{C}\mathbf{w}_1, \tag{5}$$

where $\mathbf{w}_1$ denotes the principle eigenvector (after unitization) of matrix $\mathbf{S}^T\mathbf{D}^{-1}\mathbf{S}\mathbf{C}$. The proof is finished. □

5 Inferring Matching Probability

Given the similarity function of records, a typical solution is to set a threshold and treat the record pairs with similarity score larger than the threshold as the same entity. However, tuning such similarity threshold is challenging as it is domain specific. To address this issue, we propose a new graph-theoretic method that estimates the matching probability (i.e., $p(r_i, r_j)$ in ITER) based on the similarity function. Owing to the probability nature of $p(r_i, r_j)$, the matching threshold will be much easier to tune for practitioners — usually the value is close to 1 and the setting can be universal across domains. Another desired property is that the matching probability can be fed to the ITER algorithm, serving as the edge weight in the Bipartite graph to enhance the effectiveness of learned similarity.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

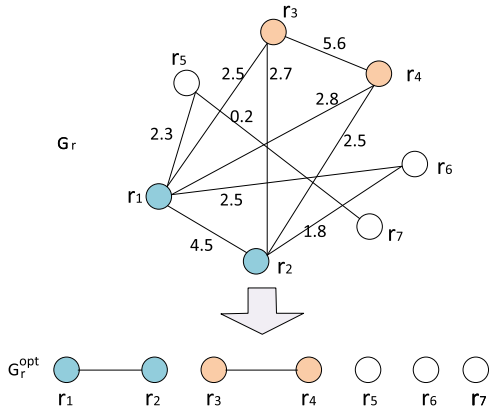


Fig. 3. An example illustrates our graph-based problem formulation: the objective is to reduce the record graph G_r to the ground-truth graph G_r^{opt}.

In the section, we first present a graph-based problem formulation for entity resolution. By interpreting the classic problem from the perspective of random walk, we propose a heuristic random surfer-based sampling (RSS) method to estimate matching probability by utilizing the topological structure of record graph. However, RSS suffers from severe efficiency problem for dense graphs. We thus propose CliqueRank to resolve the efficiency issue and render our unsupervised framework applicable in practice.

5.1 Problem Formulation

Let G_r be a record graph, where each node represents a record and the edge weight denotes the similarity of two connected records.[2] Let G_r^{opt} be the "ground-truth" record graph, in which two records are connected if and only if they refer to the same entity. It is clear that for G_r^{opt}, the records that refer to the same entity form a clique, which is disconnected with the remaining parts of G_r^{opt}. Our objective is to leverage the topological structure of G_r and transform it to G_r^{opt}.

Fig. 3 shows an illustrative example. Graph G_r is drawn from the output of ITER, where the edge weight is $s(r_i, r_j)$ obtained by Equation (2) . Graph G_r^{opt} encodes the ground-truth. As illustrated, record pair (r_1, r_2) refers to the same entity, and so is the pair (r_3, r_4); for the remaining pairs, although they are associated with a certain similarity in G_r, they are actually not matching. Through this way of representing data, the objective of entity resolution becomes that of transforming the weighted G_r to the structure of G_r^{opt}. More specifically, the task includes identifying the edges of matching pairs (i.e., (r_1, r_2) and (r_3, r_4) in Fig. 3), and removing the edges of non-matching pairs.

5.2 Random Walk Based Interpretation

To tackle the graph reduction problem, our primary idea is to leverage the *clique* structure of G_r^{opt}, similar to the transitivity property used in the crowd-sourcing methods to resolve conflicting pairs. However, instead of judging the pair similarity in an interactive fashion with crowd workers, our algorithm automatically learns the matching probability without human intervention. Ideally, if records r_i and r_j refer to different entities, they are supposed to be located in different cliques, which are unreachable from each other. On the contrary, if r_i and r_j refer to the same entity, they should be in the same clique and reachable from each other; that is to say, *if we start a random walk from one record r_i, it will be very likely to visit the other record r_j within a fixed number of steps via a rectified random walk strategy*. We leverage this key insight to develop a random surfer-based sampling (RSS) method to identify matching records.

The algorithmic framework of RSS, sketched in Algorithm 2, is simple and straightforward. After constructing the record graph with the similarity function, we perform the sampling process for each record pair (r_i, r_j). In particular, we simulate M times of random walk, half of them starting from r_i and the other half starting from r_j, and estimate the probability of r_i reaching r_j or vice versa (Lines 3–7). Each random walk outputs a number 1 or 0, indicating whether the surfer has reached the target node within a given number of S steps. The matching probability $p(r_i, r_j)$ is then estimated as the percentage of walks that have successfully reached the target (line 8). If (r_i, r_j) is a matching pair, we expect the reaching probability to be close to 1.

We consider both directions of random walk for an edge (r_i, r_j) because the reaching probability from r_i and r_j could be greatly different. An extreme example is the corner case where a node only has one neighbor, such that, the surfer from the node will always reach the target since there are no other choices. The bi-directional random walk can depress such negative cases.

In the next, we elaborate how we design the random walk process, a key component of our RSS method.

Algorithm 2. Random-Surfer Sampling (RSS)

1: Construct a record graph G_r based on $s(r_i, r_j)$;
2: **for** each edge $(r_i, r_j) \in G_r$ **do**
3: $c_1 \leftarrow 0$; $c_2 \leftarrow 0$
4: **for** $m \leftarrow 1; m \leq M/2; m++$ **do**
5: $c_1 \leftarrow c_1 + RandomWalk(r_i, r_j)$;
6: **for** $m \leftarrow 1; m \leq M/2; m++$ **do**
7: $c_2 \leftarrow c_2 + RandomWalk(r_j, r_i)$;
8: $p(r_i, r_j) \leftarrow (c_1 + c_2)/M$;
9: **return** $p(r_i, r_j)$;

Algorithm 3. RandomWalk($start$, $target$)

1: $cur \leftarrow start$;
2: **for** $s \leftarrow 1; s \leq S; s++$ **do**
3: pick a random value $b \in (0, 1)$;
4: $s'(cur, target) \leftarrow (1 + b) \cdot s(cur, target)$;
5: $next \leftarrow$ pick a node from neighbors of cur based on the new transition probability $p_b(r_i \rightarrow r_j)$;
6: **if** $next == target$ **then**
7: **return** 1;
8: **if** edge $(next, target) \notin G_r$ **then**
9: **return** 0;
10: $cur \leftarrow next$;
11: **return** 0;

Rectified Random Walk. Recall that our objective is to interpret matching probability $p(r_i, r_j)$ as the reaching

2. Note that according to our ITER algorithm, two records are connected only if they share at least one term.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

probability from r_i to r_j within a moderate number of S steps. If two records refer to the same entity, the reaching probability is close to 1. Otherwise, the probability would be close to 0. Conventional random walk cannot satisfy the requirements because it uses linear transition probability among the outgoing edges. If S is large, the running time increases dramatically and there would be many non-matching pairs with reaching probability close to 1. If S is small, we may obtain very low reaching probability of matching pairs, which are considered as false negatives. Consequently, S would become a very sensitive parameter to tune.

To resolve the issue, we propose to champion the edges of high similarity scores. Instead of using a linear transformation of edge weights, we thus devise the transition probability as non-linear transformation of edge weights

$$p(r_i \rightarrow r_j) = \frac{s(r_i, r_j)^\alpha}{\sum_{r_j \in O(r_i)} s(r_i, r_j)^\alpha}, \quad (6)$$

where $p(r_i \rightarrow r_j)$ denotes the probability of choosing r_j with the current step at r_i and $O(r_i)$ denotes the neighbors of r_i in G_r. α is a tunable parameter that controls the degree of promoting edges of high weights – the larger α is, the more likely it is to choose the high-weight edges. In practice, α is often set large enough to increase the probability gap between picking matching records as the next node and picking non-matching neighbors. In this way, the whole random walk procedure is expected to be restricted in the "groud-truth" clique until the target node is reached.

To be concrete, in our toy example of Fig. 3, if we set $\alpha = 10$, we will have $p(r_1 \rightarrow r_2) = 0.984$; as a result, the random walk from r_1 would, almost with certainty, choose r_2 as the next node (which is as expected). On the other hand, the transition probability for $r_4 \rightarrow r_1$, despite a relatively high similarity score of 2.8, would be assigned a very low probability based on our normalization strategy.

With the rectified transition probabilities, the reaching probability between two non-matching records is still very small, even after many steps of random walk. On the other hand, the similarity scores between matching pairs are generally close to each other and not affected by the normalized transition probability. After a moderate number of steps, a record is highly likely to reach its matching pair. In this way, we can use the reaching probability to estimate the matching probability.

In our implementation, besides the non-linear transition probability, we also make two other refinements. First, before choosing the neighbor to jump, we add a bonus weight to the edge between the current node and target node (lines 3-5, where b is a random number in the range (0,1)). More specifically, let r_i be the current node of random walk and r_t be the target node. The normalization factor becomes

$$norm(r_i, r_t) = (1+b)^\alpha s(r_i, r_t)^\alpha + \sum_{r_k \in O(r_i) \wedge r_k \neq r_t} s(r_i, r_k)^\alpha. \quad (7)$$

We define the weight boosted transition probability $p_b(r_i \rightarrow r_j)$ with respect to a particular random walk process from r_i to r_t as

$$p_b(r_i \rightarrow r_j) = \begin{cases} \frac{(1+b)^\alpha s(r_i, r_j)^\alpha}{norm(r_i, r_t)} & \text{if } r_j = r_t \\ \frac{s(r_i, r_j)^\alpha}{norm(r_i, r_t)} & \text{if } r_j \neq r_t \end{cases}.$$

Note that b is randomly generated for each $p_b(r_i \rightarrow r_j)$. This step is important to ensure the RSS algorithm can identify the entities that are associated with many records. Given a benchmark dataset in which the largest entity has m records, which correspond to a very large clique with m nodes in the record graph; if the surfer starts from a node of the clique, it may have to jump many steps to reach the target node, even though the target is in the same clique (as the non-linear transition probability for each outgoing edge is very small, around $1/(m-1)$ when the edge weights in the same clique are close to each other). This results in a dilemma: if we want to detect the clique by random walk, S has to be a very large number, which will significantly slow down the simulation process. By boosting the weight of edge between the current node and target node in each step, we can resolve the big-clique issue and speed up the simulation without degrading the resolution accuracy. Note that the reaching probability between non-matching pairs may also be boosted. Since the bonus factor is selected randomly between (0,1) for each step of random walk and these pairs normally have small weight, the success rate of reaching a non-matching neighbor within S steps is still quite small.

Second, we additionally apply an early stop strategy: if the surfer has reached a node that is not a neighbor of the target, we consider it a failure and return 0 (lines 8–9). This is because the surfer has traversed outside of the clique that the target node locates in, meaning that the start node and target node are less likely to refer to the same entity.

Complexity Analysis. Let n be the number of records and the number of edges in G_r is $O(n^2)$ in the worst case. Since we conduct the sampling process for all the edges for M times and the random walk needs to select the next destination from all the neighbors for at most S steps, the complexity of RSS is $O(M \cdot S \cdot n^3)$. Obviously, the cubic complexity renders RSS not applicable in practice, especially in large-scale dense G_r. In the following, we will present a more efficient matrix-based algorithm named CliqueRank to replace RSS.

5.3 CliqueRank Algorithm

The RSS algorithm is essentially designed to estimate the probability of r_i reaching r_j within S steps. With sufficient number of steps, the probability is expected to be close to 1 if r_i and r_j refer to the same entity. To revisit RSS from the perspective of matrix operations, we first introduce an $n \times n$ probability transition matrix M_t, whose entry is the probability normalized non-linearly in Equation (6)

$$M_t[i, j] = p(r_i \rightarrow r_j). \quad (8)$$

If r_i and r_j are disconnected in G_r, we set $M_t[i, j] = 0$. We also set $M_t[i, i] = 0$ such that the random walk always chooses a neighboring node to proceed.

Let $M_t^S[i, j]$ be an $n \times n$ matrix whose entry is the probability matrix of r_i reaching r_j with exactly S steps. With the transition matrix M_t, we have

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

$$M_t^S = \underbrace{M_t \times M_t \times \cdots \times M_t}_{S-1 \text{ times of operation} \times} . \quad (9)$$

When $S = 1$, the transition matrix M_t has captured the probability of r_i reaching r_j within one step and $M_t^S = M_t$. When $S > 1$, we leverage the preserved probability matrix M_t^{S-1} to calculate M_t^S. In particular, we split the random walk from r_i to r_j with S steps into two components. First, we pick a neighbor of r_i in G_r, say r_k, with probability $p(r_i \rightarrow r_k)$ preserved in M_t. For each possible r_k, its probability of reaching r_j with $S-1$ steps has been calculated in M_t^{S-1}. By multiplying these two matrices, we yield M_t^S which sums up all the probabilities of paths with S steps.

It is noticeable that we customize the standard random walk with two improvements in our RSS algorithm. One is the weight boosting strategy to resolve the issue of low reaching probability when walking within a very large clique. The other customization is to return 0 if the random walk visits a node not directly connected to the target. To address these two refinements, we define a new weight boosted matrix M_b whose entry $M_b[i,j]$ is set to $p_b(r_i \rightarrow r_j) = \frac{(1+b)^\alpha s(r_i,r_j)^\alpha}{norm(r_i,r_j)}$. Let M_n be the adjacency matrix in which $M_n[i,j] = 1$ if r_i and r_j are connected in G_r. Otherwise, $M_n[i,j] = 0$. With the matrices M_b, M_t and M_n, we revise the definition of M_t^S to

$$M_t^S = \begin{cases} M_b & \text{if } S = 1 \\ M_t \times (M_t^{S-1} \odot M_n) & \text{if } S > 1 \end{cases}.$$

The $\odot$ operation refers to pair-wise multiplication between two matrices and $\times$ is the normal matrix multiplication. When $S = 1$, the matrix is initialized to M_b. In the subsequent steps, $M_t^{S-1} \odot M_n$ sets the reaching probability of non-adjacent pairs to 0 to prevent the random walk form deviating from the groundtruth clique. The effect is equivalent to lines $8-9$ in RandomWalk algorithm. Then, matrix multiplication sums up the reaching probability with exactly S steps by reusing the results in M_t^{S-1}. Finally, the matching probability $p(r_i, r_j)$ can be derived from M_t^S. It is approximated by the probability of r_i reaching r_j within S steps. To take into account random walk from both directions, we take an average of the probabilities of $M_t^k[i,j]$ and $M_t^k[j,i]$ for all $1 \le k \le S$

$$p(r_i, r_j) = \sum_{k=1}^{S} \frac{M_t^k[i,j] + M_t^k[j,i]}{2}. \quad (10)$$

Relationship Between RSS and CliqueRank. There are two significant differences between RSS and CliqueRank. First, RSS uses sampling to approximate the reaching probability, while CliqueRank uses matrix multiplication to calculate the precise probability. This is similar to using Monte Carlo methods [31] to solve the problem of SimRank, which can sometimes achieve better efficiency compared to iterative matrix multiplication. Second, CliqueRank overestimates the probability of boosted sampling strategy. In RSS, the rectified $p_b(r_i \rightarrow r_j)$ for non-target neighbors in each step of random walk uses $norm(r_i, r_t)$ as the denominator, whereas M_t uses $\sum_{r_j \in O(r_i)} s(r_i, r_j)^\alpha$ as the denominator (see the definition of $p(r_i \rightarrow r_j)$ in Equation (6)), which is smaller than $norm(r_i, r_t)$. CliqueRank cannot use $p_b(r_i \rightarrow r_j)$ as the transition matrix because its denominator $norm(r_i, r_t)$ is determined by the concrete target node r_t in random walk. Therefore, we can conclude that the estimation of reaching probability in CliqueRank is more aggressive than RSS with the weight boosting strategy.

Complexity Analysis. Since the calculation of M_t^S involves $S-1$ times of matrix multiplication between two $n \times n$ matrices, the complexity is cubic, i.e., $O(S \cdot n^3)$. Compared to RSS, CliqueRank has two advantages. First, it can reuse the intermediate results of matrix multiplication from steps $S-1$ to S. Second, it can leverage third-party library optimized for fast and parallel matrix operation. The efficiency improvement will be evaluated in our experimental study.

TABLE 3
An Example of Wrong Reinforcement Between ITER and CliqueRank That Missed a Positive Matching

Iteration	1	2	3	4	5	6
x_t	0.79	0.49	0.36	0.33	0.02	$5e^{-5}$
$s(r_i, r_j)$	1.69	1.09	0.61	0.61	0.30	0.29
$p(r_i, r_j)$	0.995	0.983	0.009	0.001	$7e^{-5}$	$6e^{-6}$

6 Reinforcement Between ITER and CliqueRank

With the derived matching probability, we can use it to refine the ITER algorithm. As shown in Equation (1), the matching probability can play an impact on the weight assignment of terms. The weight of common terms in non-matching pairs can be further depressed by Equation (1). The weight of discriminative terms that only occur in matching pairs can be further promoted because these pairs are likely to have high similarity scores and their estimated matching probability would be close to 1. On the other hand, the correct assignment of term weight can yield appropriate similar scores for each pair of records and help identify truly matching pairs. Thus, the enhanced content-based similarity and pair matching probability can reinforce each other in our fusion framework. Our fusion framework iterates ITER and CliqueRank algorithms multiple times to refine the term weight and record similarity.

In most cases, the aforementioned interaction between ITER and CliqueRank can play a positive role to improve the accuracy of entity resolution. However, it is also possible that the interaction between ITER and CliqueRank is reinforced towards the wrong direction. Given a matching pair (r_i, r_j), if the CliqueRank assigns a low matching probability for $p(r_i, r_j)$ when the algorithm is first called, the weight of discriminative terms shared by records r_i and r_j will be further depressed in Equation (1). Consequently, this will derive a smaller similarity value $s(r_i, r_j)$ in Equation (2), which leads to an even smaller $p(r_i, r_j)$.

An example of wrong reinforcement is shown in Table 3. Given a pair of matching records "*microsoft wireless mobile mouse 4,000 sgkomen pink mice d5d-00052 20.0*" and "*microsoft wireless mobile mouse 4,000 special edition susan komen mice d5d-00052 18.57*", the most discriminative term is "d5d-00052" which refers to a product model. The term weight x_t, pair similarity $s(r_i, r_j)$ and matching probability $p(r_i, r_j)$ are

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

reported with six iterations of ITER and CliqueRank algorithms. We can see that in the first two iterations, the similarity score between these two records is high enough to obtain a matching probability close to 1. In the third iteration, the term weight decreases to 0.36, which reduces the pair similarity score to 0.61 and results in a very low matching probability due to our proposed non-linear normalization strategy in Equation (6). Finally, the pair is considered as not matching by our algorithm, leading to a case of false negative.

We propose two heuristic strategies to alleviate the above issue of false negative. First, instead of using random initialization for $p(r_i, r_j)$ when the algorithm ITER is first called, we use similarity score based on tf-idf to initialize matching probability. Tailored initialization for performance improvement has been a popular strategy in many tasks such as k-means clustering [32] and deep learning models [33]. Specially in our application, let $max\{s_{tf-idf}(r_i, r_j)\}$ be the maximum similarity score based on tf-idf. We initialize $p(r_i, r_j) = \frac{s_{tf-idf}(r_i, r_j)}{max\{s_{tf-idf}(r_i, r_j)\}}$ before we call the algorithm of ITER. For example, suppose the similarity score between r_1 and r_2 based on tf-idf is 0.225 and the maximum similarity between r_1 and the other records is 0.403, we initialize the matching probability $p(r_1, r_2)$ as $0.225/0.403 = 0.558$. Compared to random assignment of matching probability, such initialization can help reduce the risk of wrong reinforcement after the first iteration of ITER and CliqueRank algorithms.

Second, we identify the highly discriminative terms (such as telephone number, postcode, product model, or software serial number) and freeze their weight to a high value without being affected by CliqueRank. To find such terms, one possible solution is to train a binary classification model, whose input is the vectorized embedding of each word and the output is a binary label to determine the discriminativeness. However, the supervised model requires non-trivial labeling cost and does not fit our unsupervised framework. Our solution is to take advantage of the characteristics of the benchmark datasets and develop a rule with two filtering conditions. 1) We require a highly discriminative term to be shared by exactly one pair of records (r_i, r_j). The reason is that if there exist multiple records referring to the same entity, the pair similarity scores between these records are not likely to exhibit high variation. Hence, it is easier for the random-walk based CliqueRank to surf within the correct clique. If an entity is associated with only two records (r_i, r_j), as the example in Table 3, the random walk has a higher chance to reach other non-matching records, especially when these non-matching records share a considerable number of non-discriminative terms with r_i and r_j. 2) We require the term to be neither a digital number nor included in the titles of Wikipedia pages. The exclusion of digital number can help prevent considering price value terms as discriminative terms as the same product can be sold at difference prices. The other constraint is used to exclude the terms that are not discriminative in a general document corpus. For instance, "town" only appears twice in the dataset of Amazon-Google. It is obvious that the term is not truly discriminative and should be excluded. The purpose of using Wikipedia as a reference set is to avoid introducing any new parameter. We simply need to extract the terms from all titles of wikipedia pages to guide the elimination of non-discriminative terms.

7 Blocking Strategy

Blocking is an approximate technique to improve the efficiency of large-scale entity resolution. It clusters similar records into blocks and only executes comparison for record pairs within the same block. Any two records in different blocks are considered as non-matching. In this way, a small number of true matches may be missed, but the total number of pair comparison can be significantly reduced. Blocking has been widely used in machine learning based approaches because the trained binary classifier needs to go through each pair of candidate records. The two deep learning approaches, DeepER [8] and DeepMatcher [10], are also no exception. As to our unsupervised framework, we can leverage blocking to generate multiple blocks and then run ITER and CliqueRank within each block. Since the complexity of CliqueRank is cubic, the total running time can be dramatically reduced by blocking techniques.

There have been a considerable number of blocking strategies proposed in the literature. Recently, a comprehensive experimental evaluation of 17 types of state-of-the-art blocking methods was conducted by Papadakis *et al.* [34]. The main purpose is to examine the robustness of their internal configurations and the trade-off between effectiveness and time efficiency. In DeepER [8], locality-sensitive hashing (LSH) [35] is used for blocking. This is because the text records have been transformed into high-dimensional numeric vectors and LSH is a popular algorithm for approximate similarity retrieval or clustering.

In this paper, we will propose our own blocking method that is suitable for the proposed unsupervised framework. First, our method is designed to support textual data and we do not assume the existence of attribute information to facilitate blocking. Thus, approaches such as attribute clustering used in [36] is not applicable. Second, most of the blocking methods examined in [34] tend to generate a large number of small blocks. In our scenario, CliqueRank is based on random walk on the record similarity graph and it prefers a block with moderate size.

Our proposed blocking method is designed with three incentives. 1) It is non-redundant and each record appears in only one block. There are two positive side effects brought by non-redundancy: the number of candidate pairs can be further reduced and there is no need to devise an aggregation strategy for the conflicting matching results generated by different blocks. 2) It involves only one parameter, i.e., the number of blocks B. This simplifies the process of parameter tuning and fits our unsupervised framework. 3) We expect the size of blocks to be relatively balanced. This is because the complexity of CliqueRank is cubic and the running time would be dominated by the block with the largest number of records.

The detailed procedure is illustrated in Algorithm 4. Let S_A and S_B be the two datasets for entity matching. The first step is to split dataset S_A into B blocks with equal size $\frac{|S_A|}{B}$. The set C is used to mark the records that have not been assigned to a block. For each block generation, we start by picking a random seed tuple r_i and find another record $r_k \in C$ with the highest tf-idf similarity score. r_k is allocated to the same block with r_i. After that, we find another record most similar to r_k and repeat the process until we find $\frac{|S_A|}{B}$ records for the bth block. When all the records in S_A have been partitioned equally into

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

B blocks, we allocate records in S_B to the generated blocks with a similar procedure. For each record $s_b \in S_B$, we find the most similar record $r_a \in S_A$ based on tf-idf measure and assign r_b to the same block with r_a. Even though dataset S_B is not equally partitioned, the random seed record selection and the moderate, equal-size blocks in S_A still provide us with a relatively balanced partition for S_B.

Algorithm 4. Blocking Algorithm

```
C ← S_A;
for b ← 1; b ≤ B; b++ do
    pick a random seed record r_i from C;
    assign r_i to the bth block;
    for j ← 1; j < |S_A|/B; j++ do
        find r_k ∈ C with the highest tf-idf similarity to r_i;
        assign r_k to the bth block;
        C ← C \ {r_k};
        r_i ← r_k;
for each record r_b ∈ S_B do
    find r_a ∈ S_A with the highest tf-idf similarity to r_b;
    assign r_b to the same block with r_a;
```

8 Extension for Structured Record Matching

Our ITER and CliqueRank algorithms are naturally designed for textual data which is unstructured. In this section, we discuss how to further improve the matching accuracy when the dataset is structured with multiple attributes. Besides textual descriptions, these attributes could be categorical (e.g., brand name) or numerical (e.g., price).

To take advantage of the additional attribute information, we propose to extend our matching condition to integrate these attributes with the purpose of reducing false positives. More specially, we originally use the matching probability $p(r_i, r_j)$ generated by CliqueRank to determine whether records r_i and r_j refer to the same entity. If $p(r_i, r_j) > \eta$, we consider the two records are matching. To eliminate the cases of false positives, i.e., two non-matching records with high $p(r_i, r_j)$, we devise additional matching constraints based on the structured information. For categorical attributes such as brand and manufacture, we require that two matching records have to share the same category. For numeric attributes such as price, we devise a loose constraint for filtering false positives, i.e., we require that $\frac{max\{price(r_i), price(r_j)\}}{min\{price(r_i), price(r_j)\}} < 4$. This is because the price is not a very reliable indicator for entity matching because the E-commerce websites may have different pricing strategies for the same product. Thus, the integration of additional filtering conditions in fact involves human intelligence to manually devise rules to facilitate our proposed unsupervised framework. We also observe that missing data are quite common in the structured datasets and our strategy is to simply ignore them. In other words, given a pair of records r_i and r_j, we only apply the conditions on matching probability and attributes with non-empty values.

9 Experiments

We conduct comprehensive experiments to evaluate the accuracy and efficiency of our proposed unsupervised framework based on blocking and graph algorithms. In the following, we first introduce the benchmark datasets, comparison methods and experimental settings. Following that, we compare our unsupervised framework with state-of-the-art DL approaches and provide comprehensive break-down analyses to examine the effectiveness of each technical component.

TABLE 4
Statistics of the Benchmark Datasets

Dataset	Domain	#Tuples	#Matches
Abt-Buy	product	1,081 - 1,092	2,097
Walmart-Amazon	electronics	2,554 - 22,074	1,154
Amazon-Google	software	1,363 - 3,226	1,300
DBLP-ACM	bibliographic	2,616 - 2,294	2,224

9.1 Benchmark Datasets

We use the following benchmark datasets that are publicly accessible.[3,4] The statistics are summarized in Table 4 for performance evaluation. We can see that these datasets are mainly from E-commerce and bibliographic applications. Walmart-Amazon, Amazon-Google and Abt-Buy contain product information from different domains, including electronics and software. DBLP-ACM is a bibliographic dataset containing information on academic publications. The largest dataset in these two domains is Walmart-Amazon, which could generate more than 56 million candidate pairs.

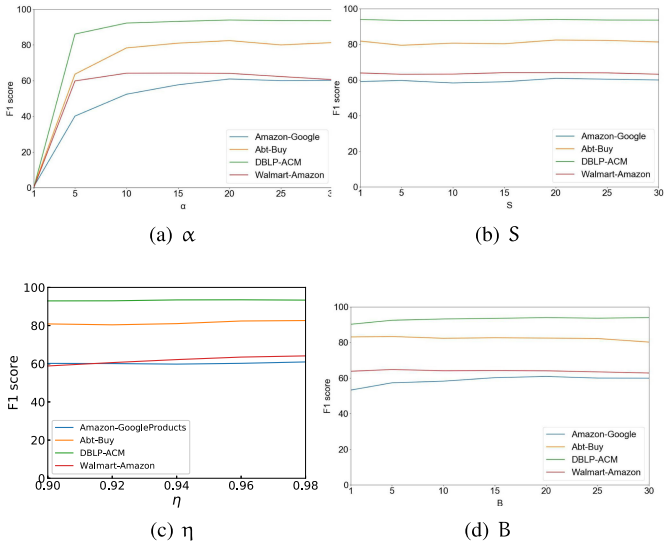
9.2 Comparison Methods

We select DeepMatcher [10] and Magellan [25] as the comparison methods because we use their provided datasets and are able to reproduce their accuracy on the benchmark datasets. Following DeepMatcher and Magellan, we define precision (P) as the fraction of match predictions that are correct, and recall (R) as the fraction of correct matches being predicted as matches. We use F_1-score as the performance metric and $F_1 = 2PR/(P + R)$. It is worth noting that DeepMatcher and Magellan belong to the category of supervised learning and they split the labelled dataset into the ratio of $3 : 1 : 1$ for the purposes of training, validation and testing, respectively. Their reported F_1-scores are evaluated from the test subset. In contrast, our proposed ITER+CliqueRank is unsupervised and it does not rely on a labelled dataset. We run ITER+CliqueRank to derive the matching probabilities of all pairs of records and evaluate F1-scores from the whole dataset.

All the experiments are carried out on a server with Xeon CPU 2.6 GHz (32 cores) and 32 GB memory, running Centos 6.5. We implemented our unsupervised algorithm with C++. In the implementation of CliqueRank, Eigen library[5] is used to boost matrix multiplication. The termination condition of the reinforcement process between ITER and CliqueRank is set to 6 iterations. In other words, we run ITER+CliqueRank for 6 iterations and use the accuracy from the last iteration to compare with baselines.

3. https://dbs.uni-leipzig.de/en/research/projects/object_matching/fever/benchmark_datasets_for_entity_resolution
4. https://sites.google.com/site/anhaidgroup/useful-stuff/data
5. http://eigen.tuxfamily.org/index.php?title=Main_Page

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.



(a) α (b) S (c) η (d) B

Fig. 4. Parameter sensitivity analysis for the algorithm of ITER +CliqueRank.

9.3 Parameter Setting

Our ITER algorithm does not involve any parameter that requires tuning. The CliqueRank algorithm has three parameters: α for the non-linear transformation in the RSS algorithm, S as the maximum number of steps for the random walk and finally the probability threshold η. In the following, we elaborate the criterion for parameter setting. α is normally set large enough to generate a dominating gap for the transition probability to matching and non-matching neighbors. S also needs to be set to a large value to identify big cliques with multiple records referring to the same entity. η is normally set to a value close to 1 as it refers to the matching probability threshold and we require a high confidence. In our implementation, we universally set $\alpha = 20$, $S = 20$ and $\eta = 0.98$ to achieve promising accuracies and run the iterations of ITER followed by CliqueRank for 6 times (the reinforcement effect will be examined in Section 9.7). This is an attractive feature because we can actually use the same parameter settings for all the three datasets from different domains. It greatly alleviates the pain of parameter tuning and exhibits the generality of our proposed framework.

We also conduct an experiment on parameter sensitivity analysis. If varying a parameter has no/little effect on the matching accuracy, it implies that the parameter can be picked from a large interval without affecting the accuracy dramatically. Such flexibility reduces the efforts on parameter tuning. The results with varying α, S, η and block number B on the four benchmark datasets are shown in Fig. 4. We can see that the F1-score in the benchmark datasets remains relatively stable as values of the four parameters vary in our CliqueRank algorithm. We also note that when η is selected from the range [0.90,0.98], the accuracy exhibits low variance. Compared with previous similarity score based approaches, our probability based matching criterion requires fewer tuning efforts and is more effective and robust.

TABLE 5
F1-Scores of Clique+ITER (Unsupervised), DeepMatcher (Supervised), and Magellan (Supervised) in the Benchmark Datasets

	Clique+ITER	DeepMatcher [10]	Magellan [25]
Abt-Buy (U)	**82.5**	62.8	43.6
Walmart-Amazon (U)	**64.1**	61.7	-
Amazon-Google (U)	61	65.1	-
DBLP-ACM (U)	94.0	97.8	-
Walmart-Amazon (S)	66.3	66.9	71.9
Amazon-Google (S)	61.5	69.3	49.1
DBLP-ACM (S)	95	98.4	98.4

The symbols (S) and U denote structured and unstructured datasets, respectively.

TABLE 6
Efficiency of Iter+CliqueRank Without Blocking

	Amazon-Google	Abt-Buy	DBLP-ACM
#nodes in G_r	4,589	2,173	4,910
#edges in G_r	1,322,692	855,170	3,331,734
Time for ITER	18s	6s	30s
Time for CliqueRank	31.2m	5.6m	42.5m
Speedup v.s. RSS	3.5x	1.6x	1.9x

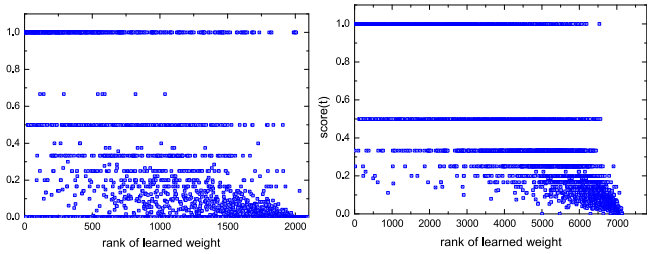
9.4 Accuracy in Benchmark Datasets

We compare our unsupervised framework with DeepMatcher and Magellan and report the F1-scores in Table 5. Since DeepMatcher can support both unstructured data with only textual information (such as Abt-Buy) and structured data with additional attribute information (such as Walmart-Amazon, Amazon-Google and DBLP-ACM), we conduct the experiments on both types of datasets. We convert the three structured datasets into their unstructured counterparts by removing the attribute information. The symbol (U) represents an unstructured dataset and (S) refers to a structured dataset. We can see that the accuracy in the structured datasets is generally higher than that in the datasets with only textual information. This is because they have additional attribute information to leverage. In the textual datasets, our unsupervised framework does not require training/labelled data and achieves higher accuracy than the two competitors in the datasets of Abt-Buy and Walmart-Amazon, verifying the effectiveness of our proposed term weight calculation by ITER and matching probability estimation by CliqueRank. In the other unstructured datasets, we also yield comparable performance with DeepMatcher. In the structured datasets, the accuracy of our unsupervised framework can be further improved by the proposed heuristic rules. The number of false positives is marginally reduced and the overall F1-score increases.

9.5 Efficiency Analysis

Since our method is unsupervised, it is not fair to directly compare the running time with DeepMatcher and Magellan. First, they are supervised approaches and their running time is calculated by employing the trained model on the pair records in the test dataset. In contrast, our method can directly return the matching probabilities for all the pairs in the whole dataset and does not rely on the size of test dataset. Second, DeepMatcher is based on deep learning and prefers GPU environment. Magellan and our method run in

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.



(a) Amazon-Google (b) DBLP-ACM

Fig. 5. Visualization of the effectiveness of the learned term weights.

TABLE 7
Top-Ranked Terms by ITER in the Benchmark Datasets

Abt-Buy	mb13blkseev2, psc4000, sx10, dmcfs3s, dscw150r,19lv505, tcr13, 22lv506, 15lv505, sdyd250
Walmart-Amazon	ju-sa0912-s1, sa-309, cb-dp0b11-s1, ju-mr0012-s1, pex1pci1r, 215-sheet, jj-pcm012-s3, ktm-tp9828, jk-wr0612-s1, 12plpv3-u2k
Amazon-Google	t74-01215, csa-b250-dtop-k9, cd26xm-ch, wd-1000-0900, cd28-esk9, wg017196, xsr-18xx-vpn-fw, rhf0211us, icopydvds2, 213076-b26
DBLP-ACM	zhengying, pappas, cheetah, plug-and-play, zahir, thanisch, sukho, hyoseop, singh, chung-min

the CPU environment. Therefore, in this experiment, our goal is to evaluate the speedup brought by CliqueRank when compared with random-walk based RSS algorithm. We compare the running time of ITER+CliqueRank and ITER+RSS, without using blocking method (i.e., $B = 1$). We derive three observations from the results in Table 6. First, the complexity of ITER algorithm is linear to edge number. It runs fast as our datasets only contain at most tens of thousands of records. Its running time can be omitted when compared to RSS and CliqueRank, which is the bottleneck of our unsupervised framework. Second, the time cost of CliqueRank in each dataset is less than an hour, showing that our unsupervised method is efficient in practice and has the potential to resolve datasets with larger scale. It's also worth noting that its running time is more sensitive to the number of nodes in G_r than the number of edges. Third, CliqueRank can significantly boost the performance in dense G_r. For example, it achieves 3.5 times speedup in the Amazon-Google dataset. It uses iterative matrix multiplication to calculate the reachability probability within S steps. The intermediate results of reaching probability for $S-1$ steps, i.e., M_t^{S-1}, can be reused to calculate M_t^S. In contrast, RSS conducts random-walk based sampling for each pair independently.

9.6 Effectiveness of ITER Algorithm

We also study the efficacy of the learned term weight of the ITER method. Since our primary objective is to detect the most discriminative terms in an application domain and assign them with high weights, we use the following criterion to measure the effectiveness of a term t: $score(t) = \frac{\sum_{t \in r_i \wedge t \in r_j} I(r_i, r_j)}{P_t}$,where P_t is the number of record-pairs connected with term t in the bipartite graph and $I(r_i, r_j) = 1$ if its associated pair (r_i, r_j) refer to the same entity. Otherwise, we set $I(r_i, r_j) = 0$. Intuitively, if all the records in the inverted list of term t refer to the same entity, then t is said to be highly discriminative and assigned with score 1. In contrast, if t is a common term and occurs in many different entities, then t should be assigned with a lower score.

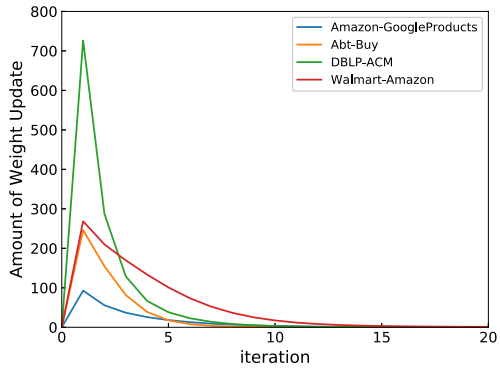


Fig. 6. Convergence of ITER algorithm.

In the optimal scenario, the ranking order of terms sorted by the learned weight x_t should follow the same order as $score(t)$. To examine this, we plot in Fig. 5 the terms in decreasing order of their weight in the x-axis. In other words, the term t with the ith highest x_r is plotted at the position of $x = i$. The y-axis value for each term t is set to $score(t)$. We can see that in the dataset of DBLP-ACM, the highly discriminative terms with $score(t) = 1$ are clustered in the front part of the x-axis. It means our ITER algorithm also considers them as much more important terms and assigns them with higher weight. The common terms that are not discriminative (with small $score(t)$) are clustered in the bottom-right corner, which is also in accordance with our unsupervised entity-term ranking strategy. The distribution pattern in Amazon-Google is not as clear as that in DBLP-ACM due to a considerable amount of wrong prediction for pair matching probability. As reported in Table 5, Amazon-Google is a much more challenging dataset than DBLP-ACM for the task of entity resolution and the accuracy is less than 70 percent for all existing methods. The wrong derivation of matching probability would affect the ITER algorithm for term weight estimation.

To empirically show that the node weight will converge, we plot the sum of weight value update of x_t in Fig. 6 for the first 20 iterations. We can see that in the initial stage the weights change dramatically, exhibiting a sharp-peek pattern for all the four datasets. This is because the parameters are initialized randomly. With a few more iterations, they can quickly reach convergence.

At the end, we depict examples of top-ranked terms in each dataset in Table 7. We can see that in the product datasets, the models are considered as highly discriminative; while in the academic domain, rare author names (e.g., thanisch) are often assigned with the highest scores.

9.7 Effect of Reinforcement

In Section 6, we propose two heuristics strategies to alleviate the issue of false negative matching caused by the reinforcement process. One is to initialize the matching probability using tf-idf similarity score, denoted by R_1 in Table 8. The

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

TABLE 8
Effect of Reinforcement Optimization Based on the Two Proposed Heuristic Strategies

	w/o R_1R_2	w/ R_1	w/ R_2	w/ R_1R_2
Walmart-Amazon	58.6	61.6	63.8	64.1
Amazon-Google	58.2	58.6	59.7	61.0
Abt-Buy	77.3	77.7	81.7	82.5
DBLP-ACM	93.1	93.3	93.8	94.0

TABLE 9
Effect of Reinforcement Between ITER and CliqueRank

Iter	Abt-Buy	Walmart-Amazon	Amazon-Google	DBLP-ACM
1	64.33	52.53	53.72	93.19
2	79.86	63.84	59.87	93.75
3	81.22	63.88	60.98	93.81
4	81.57	63.82	61.02	93.82
5	82.11	64.36	60.84	93.91
6	82.46	64.12	60.95	93.99

other is to identify the highly discriminative terms and freeze their weight without being affected by the reinforcement, denoted by R_2 in Table 8. Here, we evaluate the effect of these two heuristic strategies. From the results in Table 8, we observe positive improvement in the four benchmark datasets and R_2 plays a more important effect than R_1. Additionally, in the three E-commerce datasets, the improvement is more significant because the highly discriminative terms such as product models play an important role for entity matching. In contrast, there are much fewer highly discriminative terms in DBLP-ACM (such as authors with uncommon family names), leading to slight improvement.

Table 9 shows the effect of reinforcement with multiple iterations between ITER and CliqueRank. In the first iteration, the weight $p(r_i, r_j)$ in ITER algorithm is initialized according to the normalized tf-idf similarity score. Then, we apply CliqueRank on the derived record similarity graph G_r and obtain F1-scores listed in the first row. Thereafter, we feed the matching probability $p(r_i, r_j)$ to the bipartite graph and run ITER in the second iteration. We can see that there is a noticeable improvement of F1-scores for CliqueRank in the datasets of Abt-Buy, Walmart-Amazon and Amazon-Google. With more iterations, the accuracy exhibits an increasing trend. In the bibliographic dataset of DBLP-ACM, the tf-idf based similarity performs well and the effect of reinforcement is not as discriminative as in the other datasets. Note that there may be slight fluctuation as in these datasets. This phenomenon is similar to the decreasing trend of loss values in supervised training of machine learning models.

9.8 Effect on Blocking

The final experiments evaluate the effect of our proposed non-duplicate blocking strategy. We use the methods examined in the VLDB experimental work [34] as the comparison approaches. The implementation is available online.[6] We use Amazon-Google as the dataset and report multiple performance metrics in Table 10. PC (pairs completeness), PQ (pairs quality) and RR (reduction ratio) are three related metrics commonly used to measure blocking effectiveness. Let N_1 be the number of matching pairs in the original dataset and N_2 be the number of matching pairs in the derived blocks, we have $PC = N_2/N_1$ which is essentially the upper bound for the accuracy of ER algorithms. Let N_3 be the total number of pair comparison in the derived blocks, we have $PQ = N_2/N_3$ which measures the portion of non-redundant comparison. Let N_4 be the total number of candidate pairs in the original dataset, $RR = 1 - N_3/N_4$ measures the portion of comparison avoided by blocking algorithms. The running time column in the format of $X + Y$ refers to the blocking generation time (X in seconds) and the entity resolution time (Y in minutes) with ITER+CliqueRank.

TABLE 10
Effect of Blocking in the Dataset Amazon-Google

Method	#Block	PC	PQ	RR	Time (s)	F_1-score
StBl	2,109	90.54	12.84	99.79	$0.1s + 41m$	14.64
ACl	2,157	84.23	15.28	99.84	$1.3s + 40m$	11.79
CaCl	1,035	66.31	36.31	99.95	$1.9s + 32m$	1.22
SoNe	29,115	80.23	2.30	98.96	$0.3s + 44m$	30.92
QGBl	3,882	95.00	5.31	99.47	$0.2s + 43m$	33.46
ESuAr	7,970	93.31	3.31	99.16	$0.4s + 45m$	43.30
ECaCl	1,363	85.92	6.22	99.59	$0.6s + 42m$	44.54
EQGBl	5,004	93.15	7.15	99.61	$0.5s + 42m$	29.48
ESoNe	3,720	95.77	5.52	99.48	$0.4s + 43m$	32.82
SuAr	1,280	51.15	26.29	99.94	$0.1s + 29m$	0.46
TyMa	2,972	64.23	1.47	98.71	$0.9s + 53m$	28.32
Ours	20	94.15	0.56	95.00	$0.9s + 3m$	60.95

From the results, we can see that the blocking time of these algorithms is negligible when compared to the total running time for ER. Previous blocking methods result in a large number of blocks as they mainly use tokens or n-grams as the hashing keys to generate blocks. The redundancy can be further reduced by the *block cleaning* and *comparison cleaning* techniques examined in [34]. Overall, the objective of previous blocking methods is to achieve a good balance between PC, PQ and RR, i.e., avoid missing matches to obtain high PC but reduce unnecessary pair comparison as much as possible to improve PC and RR. Guided by the objective, they tend to generate a large number of small blocks. However, these blocking methods are not suitable for our proposed ITER+CliqueRank. As shown in the table, they result in longer running time and lower accuracy when ITER+CliqueRank is applied on their generated blocks for entity resolution. This is mainly because the graphs generated from small blocks are not informative to preserve sufficient context information for us to identify discriminative terms. Also, the default setting of S may not be a good choice for small blocks. CaCl and SuAr are two extreme cases of small blocks. In order to obtain high values of PQ and RR, they delete pair comparison aggressively and generate very small blocks, resulting in very small F_1-score when we apply ITER+CliqueRank. Our proposed blocking algorithm works well with ITER+CliqueRank because it generates a small number of moderate-size blocks

6. http://sourceforge.net/projects/erframework

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

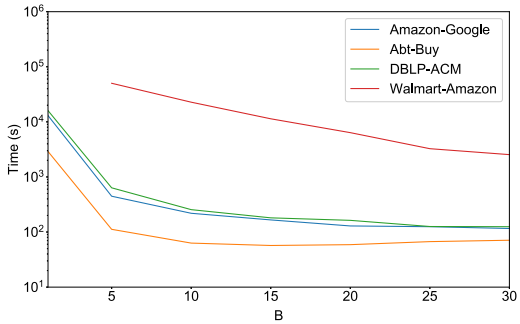


Fig. 7. Total running time with different number of blocks.

TABLE 11
Scalability of Iter+CliqueRank With $B = 20$

Datasets	10%	20%	40%	60%	80%	100%
Abt-Buy	37s	38s	41s	46s	53s	62s
Amazon-Google	51s	76s	91s	104s	117s	135s

with high PC. The block size (i.e., the number of records in a block) is in the range [179,302], with 229 being the mean number. With the balanced blocking strategy and the use of Eigen library to boost matrix multiplication within each block, the total running is around 3 minutes to handle Amazon-Google dataset.

We also evaluate the efficiency in terms of increasing number of blocks. The experiments are conducted in the four datasets with B varying from 1 to 30. Note that the F_1-scores with increasing B have been reported in Fig. 7. When $B = 1$, it refers to the running time when blocking strategy is not applied. We can see that blocking can dramatically reduce the running time. When the datasets are partitioned into 5 blocks, the efficiency is improved by more than an order of magnitude. This allows us to handle large-scale datasets without sacrificing matching quality. For instance, the running time in Walmart-Amazon when $B = 1$ is not reported because it requires at least multiple days to finish the job. When $B = 5$ is used, our ITER+CliqueRank is able to process the dataset within 13 hours. The efficiency continues to improve as B increases to 30.

To examine the scalability with input size, we sample the subsets of Abt-Buy and Amazon-Google with increasing proportion. The block number is fixed as 20 and the running time results (in seconds) are presented in Table 11. Even though in the worst case the complexity of ITER+CliqueRank is cubic w.r.t. the input size, we can see that ITER+CliqueRank scales well, due to the following reasons. First, in these real datasets, the record similarity graph is sparse and the matrix operations in CliqueRank algorithm can be well optimized by the third-party Eigen library. Second, the blocking mechanism is also helpful to suppress the growing speed of running time. Finally, since the labelling cost is expensive, the current available datasets are not very large-scale, with thousands of records. When the input size is very small, the increasing pattern of running time is not obvious (e.g., from 10 to 20 percent in Abt-Buy). When the dataset becomes much larger-scale, we can see that the running time increases more dramatically.

10 Conclusion

In this paper, we proposed an unsupervised graph-theoretic fusion framework for entity resolution. In particular, we devised two novel algorithms ITER and CliqueRank, one for term-based similarity and the other for topological confidence, that can reinforce each other. Efficiency is further improved with our proposed blocking strategy. Experimental results on four benchmark datasets verified the effectiveness of our framework. Without any labelled data, its accuracy can be comparable or even superior to the state-of-the-art deep learning based approaches.

It is noticeable that our model is sensitive to noisy terms. In our future work, we will incorporate data cleaning as a pre-processing step to improve data quality. For instances, misspelling terms will be corrected and inconsistent data format (such as "iphone 6" and "iphone-6") will be unified.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (No. 61932001 and 61702016), and a MoE grant (No. T1251RES1913) in Singapore.

References

[1] A. E. Monge and C. Elkan, "The field matching problem: Algorithms and applications," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 1996, pp. 267–270.
[2] W. W. Cohen, "Data integration using similarity joins and a word-based information representation language," *ACM Trans. Inf. Syst.*, vol. 18, no. 3, pp. 288–321, 2000.
[3] M. Bilenko, R. J. Mooney, W. W. Cohen, P. Ravikumar, and S. E. Fienberg, "Adaptive name matching in information integration," *IEEE Intell. Syst.*, vol. 18, no. 5, pp. 16–23, Sep./Oct. 2003.
[4] S. Sarawagi and A. Bhamidipaty, "Interactive deduplication using active learning," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 2002, pp. 269–278.
[5] P. D. Ravikumar and W. W. Cohen, "A hierarchical graphical model for record linkage," in *Proc. Conf. Uncertainty Artif. Intell.*, 2004, pp. 454–461.
[6] M. Bilenko and R. J. Mooney, "Adaptive duplicate detection using learnable string similarity measures," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 2003, pp. 39–48.
[7] A. Arasu, M. Götz, and R. Kaushik, "On active learning of record matching packages," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2010, pp. 783–794.
[8] M. Ebraheem, S. Thirumuruganathan, S. R. Joty, M. Ouzzani, and N. Tang, "Distributed representations of tuples for entity resolution," *Proc. VLDB Endowment*, vol. 11, no. 11, pp. 1454–1467, 2018.
[9] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
[10] S. Mudgal *et al.*, "Deep learning for entity matching: A design space exploration," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 19–34.
[11] J. Wang, T. Kraska, M. J. Franklin, and J. Feng, "CrowdER: Crowdsourcing entity resolution," *Proc. VLDB Endowment*, vol. 5, no. 11, pp. 1483–1494, 2012.
[12] S. E. Whang, P. Lofgren, and H. Garcia-Molina, "Question selection for crowd entity resolution," *Proc. VLDB Endowment*, vol. 6, no. 6, pp. 349–360, 2013.
[13] J. Wang, G. Li, T. Kraska, M. J. Franklin, and J. Feng, "Leveraging transitive relations for crowdsourced joins," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 229–240.
[14] N. Vesdapunt, K. Bellare, and N. N. Dalvi, "Crowdsourcing algorithms for entity resolution," *Proc. VLDB Endowment*, vol. 7, no. 12, pp. 1071–1082, 2014.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.

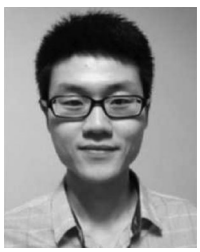
[15] S. Wang, X. Xiao, and C. Lee, "Crowd-based deduplication: An adaptive approach," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1263–1277.
[16] C. Chai, G. Li, J. Li, D. Deng, and J. Feng, "Cost-effective crowdsourced entity resolution: A partial-order approach," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2016, pp. 969–984.
[17] D. Zhang, L. Guo, X. He, J. Shao, S. Wu, and H. T. Shen, "A graph-theoretic fusion framework for unsupervised entity resolution," in *Proc. IEEE Int. Conf. Data Eng.*, 2018, pp. 713–724.
[18] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 1, pp. 1–16, Jan. 2007.
[19] W. W. Cohen, P. D. Ravikumar, and S. E. Fienberg, "A comparison of string distance metrics for name-matching tasks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2003, pp. 73–78.
[20] E. S. Ristad and P. N. Yianilos, "Learning string-edit distance," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 5, pp. 522–532, May 1998.
[21] M. Bilenko and R. J. Mooney, "Learning to combine trained distance metrics for duplicate detection in databases," *Artif. Intell. Lab.*, Tech. Rep. AI 02-296, Univ. Texas at Austin, Austin, TX, Feb. 2002.
[22] S. Tejada, C. A. Knoblock, and S. Minton, "Learning object identification rules for information integration," *Inf. Syst.*, vol. 26, no. 8, pp. 607–633, 2001.
[23] W. W. Cohen and J. Richman, "Learning to match and cluster large high-dimensional data sets for data integration," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 2002, pp. 475–480.
[24] K. Bellare, S. Iyengar, A. G. Parameswaran, and V. Rastogi, "Active sampling for entity matching," in *Proc. Int. Conf. Knowl. Discov. Data Mining*, 2012, pp. 1131–1139.
[25] P. Konda *et al.*, "Magellan: Toward building entity matching management systems," *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 1197–1208, 2016.
[26] G. Demartini, D. E. Difallah, and P. Cudr é-Mauroux, "ZenCrowd: Leveraging probabilistic reasoning and crowdsourcing techniques for large-scale entity linking," in *Proc. Int. Conf. World Wide Web*, 2012, pp. 469–478.
[27] J. Fan, Z. Wei, D. Zhang, J. Yang, and X. Du, "Distribution-aware crowdsourced entity collection," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 7, pp. 1312–1326, Jul. 2019.
[28] D. Zhang, L. Nie, H. Luan, K. Tan, T. Chua, and H. T. Shen, "Compact indexing and judicious searching for billion-scale microblog retrieval," *ACM Trans. Inf. Syst.*, vol. 35, no. 3, pp. 27:1–27:24, 2017.
[29] F. Rousseau and M. Vazirgiannis, "Graph-of-word and TW-IDF: new approach to ad hoc IR," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2013, pp. 59–68.
[30] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 3rd ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 1996.
[31] D. Fogaras and B. Rácz, "Scaling link-based similarity search," in *Proc. Int. Conf. World Wide Web*, 2005, pp. 641–650.
[32] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. Annu. ACM-SIAM Symp. Discrete Algorithms*, 2007, pp. 1027–1035.
[33] I. Sutskever, J. Martens, G. E. Dahl, and G. E. Hinton, "On the importance of initialization and momentum in deep learning," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1139–1147.
[34] G. Papadakis, J. Svirsky, A. Gal, and T. Palpanas, "Comparative analysis of approximate blocking techniques for entity resolution," *Proc. VLDB Endowment*, vol. 9, no. 9, pp. 684–695, 2016.
[35] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. Annu. ACM Symp. Theory Comput.*, 1998, pp. 604–613.
[36] G. Papadakis, E. Ioannou, T. Palpanas, C. Niederée, and W. Nejdl, "A blocking framework for entity resolution in highly heterogeneous information spaces," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 12, pp. 2665–2682, Dec. 2013.



Dongxiang Zhang received the BSc degree from Fudan University, Shanghai, China, in 2006, and the PhD degree from the National University of Singapore, Singapore, in 2012. He is now a research professor with Zhejiang University, China. He worked as a research fellow with the NeXT Research Center in Singapore from 2012 to 2014, and he was promoted to senior research fellow in 2015. From 2016-2019, he was a professor with the University of Electronic Science and Technology of China. His current research interests include smart databases and big data analytics.



Dongsheng Li received the BSc (honors) and PhD (honors) degrees in computer science from the College of Computer Science, National University of Defense Technology, Changsha, China, in 1999 and 2005, respectively. He was awarded the prize of National Excellent Doctoral Dissertation of PR China by the Ministry of Education of China, in 2008. He is now a full professor with the National Lab for Parallel and Distributed Processing, National University of Defense Technology, China. His research interests include parallel and distributed computing, cloud computing, computer network, and large-scale data management.



Long Guo received the bachelor of computer science degree from the HuaZhong University of Science and Technology, Wuhan, China, and the PhD degree from the National University of Singapore, Singapore. He is a senior algorithm engineer with Alibaba Inc. His research interests include machine and deep learning, natural language processing, and spatial data mining.



Kian-Lee Tan (Senior Member, IEEE) received the PhD degree in computer science from the National University of Singapore (NUS), Singapore, in 1994. He is a professor of computer science with the School of Computing, NUS. His current research interests include query processing and optimization in multiprocessor and distributed systems, database performance, data analytics, and database security. He has published more than 300 research articles in international journals and conference proceedings, and coauthored several books/monographs. He was a recipient of the NUS Outstanding University Researchers Award, in 1998, and the NUS Graduate School (NGS) Excellent Mentor Award, in 2011. He was a co-recipient of Singapore's President Science Award, in 2011. He is also a 2013 IEEE Technical Achievement Award recipient. He is a member of the VLDB Endowment Board and PVLDB Advisory Committee. He is an associate editor of the *World Wide Web Journal*. He has also served in the editorial board of the *Very Large Data Base (VLDB) Journal* (associate editor: 2007–2009; editor-in-chief: 2009–2015) and the *IEEE Transactions on Knowledge and Data Engineering* (2009–2013). He is a member of the ACM and senior member of the IEEE CS.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: University of Maryland College Park. Downloaded on March 01,2024 at 23:54:43 UTC from IEEE Xplore. Restrictions apply.